

Chain-of-Thought Distillation for Mathematical Reasoning

STAT GR5293 GenAI Course Project Final Report

Ian Wang

Junyang Li

Yuzhi Yao

May 2026

Link: <https://github.com/jl7230-pixel/STAT5293-GenAI-Course-Project>

Abstract

This project looks at whether a smaller language model can learn math reasoning from teacher-generated chain-of-thought examples without having to produce very long explanations. We use GSM8K as the main dataset and compare three training formats: answer-only, verbose chain-of-thought, and compressed chain-of-thought. The teacher model is GPT-4.1-mini, and the student model is Qwen2.5-3B-Instruct fine-tuned with LoRA. Our main comparison uses answer accuracy, average latency, and output length. In the pilot evaluation, answer-only training is very fast but weak in accuracy. Verbose CoT gives the best accuracy, but it is slow and produces long responses. Compressed CoT gives a close accuracy result while cutting both latency and output length. Overall, the experiment suggests that compressed reasoning is a practical compromise for small reasoning models.

Introduction

Many language models perform better on math word problems when they write out intermediate reasoning steps. This is the basic idea behind chain-of-thought prompting. For a problem like GSM8K, the model often needs to identify numbers, choose operations, and combine several steps before reaching the final answer. Showing the reasoning process can make this easier.

The problem is that full chain-of-thought is not free. Long explanations take more time to generate, use more tokens, and can be harder to deploy in small or latency-sensitive systems. In an education setting, for example, a student may not need a long paragraph for every arithmetic problem. A short explanation with the essential steps may be enough.

Our project studies this trade-off through chain-of-thought distillation. We train the same student model using three different kinds of supervision. The answer-only version gives only the final answer. The verbose CoT version gives a full step-by-step explanation. The compressed CoT version keeps the important calculation steps but removes much of the extra wording. We then compare how these choices affect accuracy, latency, and output length.

We use GSM8K as the benchmark, GPT-4.1-mini as the teacher model, and Qwen2.5-3B-Instruct as the student model. Since full fine-tuning would be expensive, we use LoRA with 4-bit quantization. The goal is not to claim a final state-of-the-art result. Instead, the goal is to build a working pipeline and see whether compressed reasoning looks promising in a small but reproducible experiment.

Research Questions

The project is organized around four questions:

1. Does reasoning supervision help the student model more than answer-only supervision?
2. How does the length of the teacher reasoning trace affect accuracy, latency, and output length?
3. Can compressed CoT keep most of the benefit of verbose CoT while being faster?
4. What problems appear when building this pipeline with limited training data and compute?

These questions are useful because real systems usually need a balance. A model that is slightly more accurate but much slower may not be the best choice for deployment. Likewise, a very fast model that cannot reason reliably is not useful for math tutoring.

Related Work

Chain-of-thought prompting was introduced to improve reasoning by asking language models to write intermediate steps before giving the final answer [1]. This is especially relevant for arithmetic word problems, where the answer depends on several smaller computations. GSM8K is a common benchmark for this type of reasoning [2].

This project is also related to knowledge distillation. In a distillation setup, a smaller student model learns from outputs produced by a stronger teacher model. For reasoning tasks, the teacher output can include both the final answer and the explanation. Work such as *Distilling Step-by-Step* shows that rationales can help smaller models learn more effectively [4].

LoRA is important for the implementation. Instead of updating all model weights, LoRA adds small trainable low-rank matrices to selected layers [3]. This makes fine-tuning a 3B model more realistic for a course project. Our project combines these ideas and asks a more practical question: if reasoning traces are useful, how long do they need to be?

Methodology

Project Pipeline

The project starts from GSM8K questions and gold answers. A teacher model generates two types of reasoning traces: verbose and compressed. We then build three dataset formats, fine-tune one student model for each format, and evaluate the resulting models. Figure 1 summarizes the workflow.

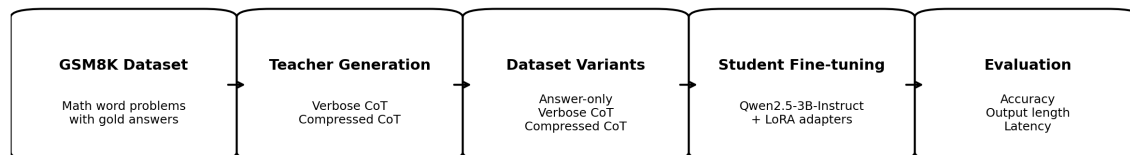


Figure 1: Project pipeline for chain-of-thought distillation.

Dataset Construction

The updated version of the notebook uses 1,000 training examples, 100 validation examples, and 100 test examples. GPT-4.1-mini is used to generate the teacher reasoning. After generation, the records are filtered so that incomplete examples are removed. The cleaned files are saved as JSONL files for the three training conditions.

Table 1: Dataset split used in the updated project pipeline.

Split	Number of Examples
Training	1,000
Validation	100
Test	100

Supervision Formats

The three formats are shown in Table 2. They differ only in what the student model is asked to learn as the target output.

Table 2: Three supervision formats used for student fine-tuning.

Condition	Description
Answer Only	The target contains only the final answer, such as Answer: 18.
Verbose CoT	The target contains a detailed step-by-step solution followed by the final answer.
Compressed CoT	The target contains a shorter reasoning trace followed by the final answer. It keeps the main arithmetic logic but removes extra explanation.

Student Model and Fine-Tuning Setup

The student model is **Qwen/Qwen2.5-3B-Instruct**. The teacher model is GPT-4.1-mini. We use 4-bit NF4 quantization and LoRA so that fine-tuning can be run with limited GPU memory. The main settings are listed in Table 3.

Table 3: Fine-tuning configuration.

Component	Setting
Student model	Qwen/Qwen2.5-3B-Instruct
Teacher model	GPT-4.1-mini
Quantization	4-bit NF4
PEFT method	LoRA
LoRA rank	$r = 16$
LoRA alpha	32
LoRA dropout	0.05
Target modules	q_proj, k_proj, v_proj, o_proj
Training conditions	answer-only, verbose CoT, compressed CoT
Training steps	50 steps per condition

Evaluation Protocol

For evaluation, each fine-tuned model generates an answer for the same test questions. We use greedy decoding. A simple regex-based function extracts the final numeric answer from the model output, and the extracted answer is compared with the GSM8K gold answer.

We record three metrics:

- **Accuracy:** the share of examples where the extracted answer matches the gold answer.
- **Average latency:** the average generation time per example.
- **Average output length:** the average length of generated responses.

The pilot evaluation uses 20 examples for each condition. This is a small evaluation set, but it is enough to see the main pattern across the three supervision choices.

Results

Teacher Data Quality

Before fine-tuning the student model, we first compare the teacher outputs. The teacher’s verbose solutions are much longer than the compressed ones, as shown in Figure 2.

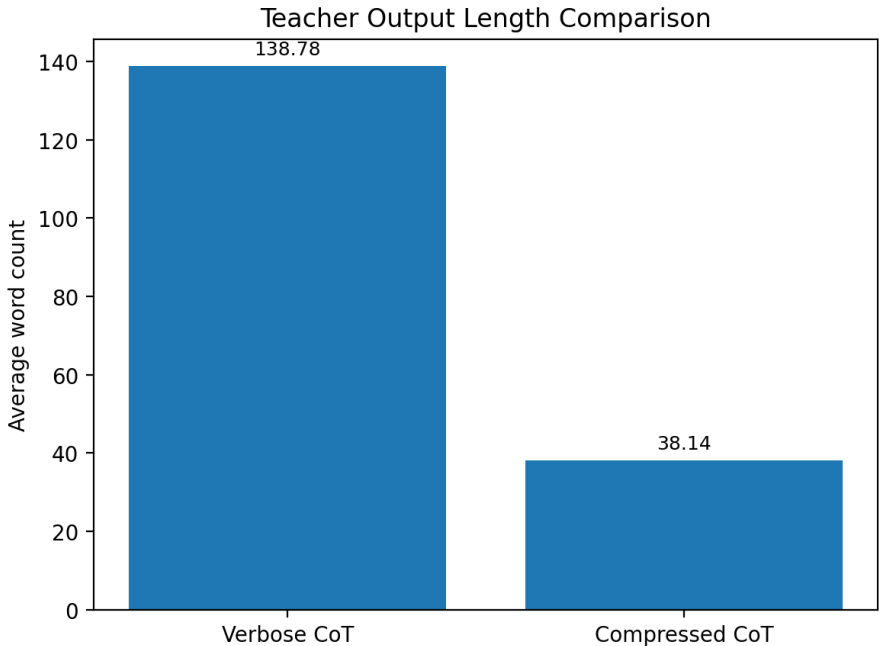


Figure 2: Teacher output length comparison between verbose and compressed CoT.

The average verbose teacher output is about 138.79 words, while the compressed output is about 38.14 words. This is a large reduction. If the compressed version still keeps the answer mostly correct, then it may be a better target for training.

Figure 3 shows the teacher answer match rate. Verbose CoT matches the gold answer in 191 out of 200 examples, while compressed CoT matches in 188 out of 200 examples.

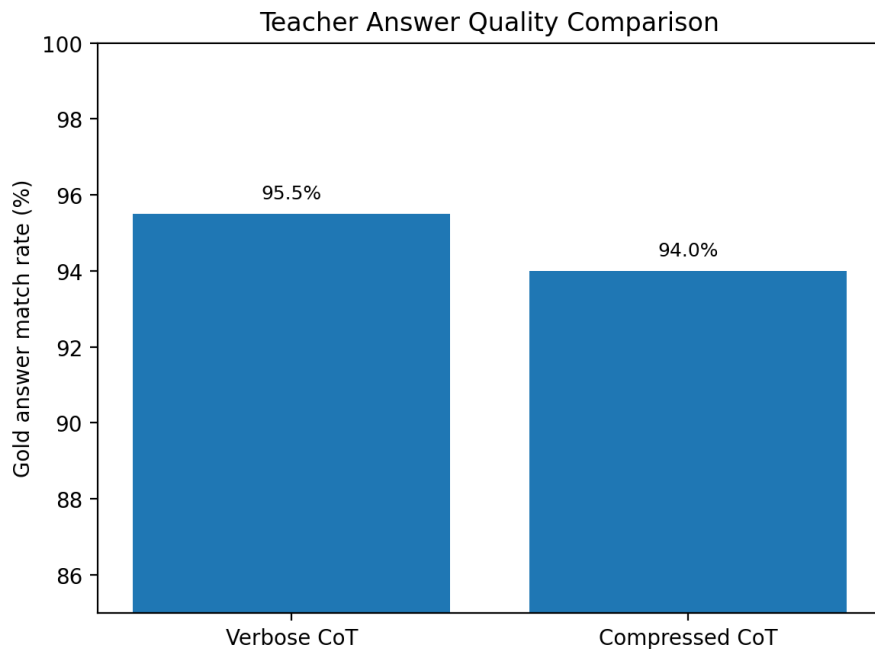


Figure 3: Teacher answer match rate comparison.

The match rate drops only slightly after compression. This supports the idea that the compressed traces are not simply losing the core solution. They are shorter, but they still usually preserve the key reasoning needed to reach the answer.

Training Behavior

The loss curves give a basic check that the fine-tuning process is working. Figure 4 shows the training and validation loss for the compressed condition.

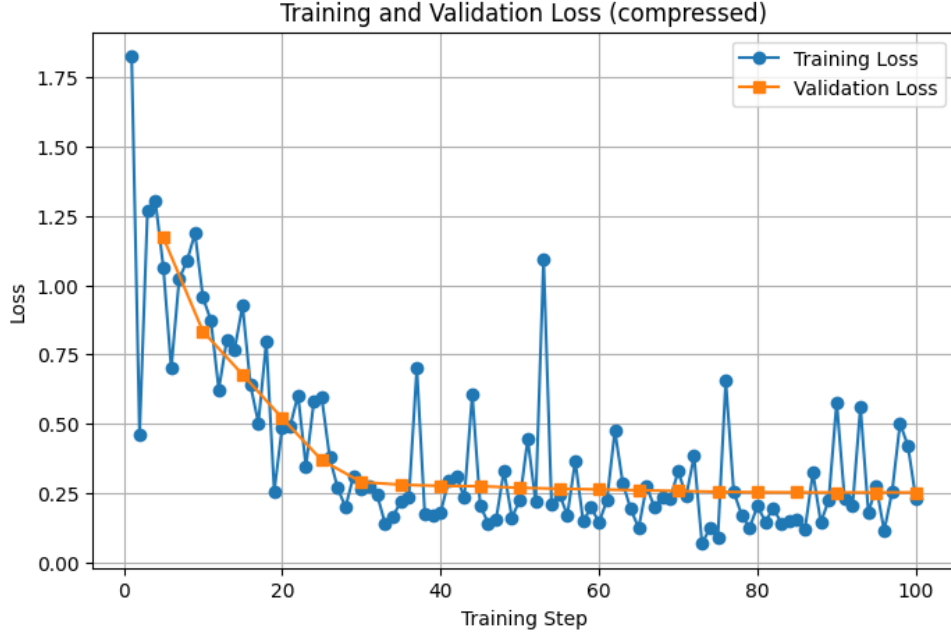


Figure 4: Training and validation loss for the compressed CoT condition.

The curve is not perfectly smooth, but this is expected because the run is short and the dataset is small. Figure 5 compares all three conditions. The answer-only condition starts with higher loss, while the CoT conditions are more stable. Figure 5 uses a moving average to make the trend easier to see.

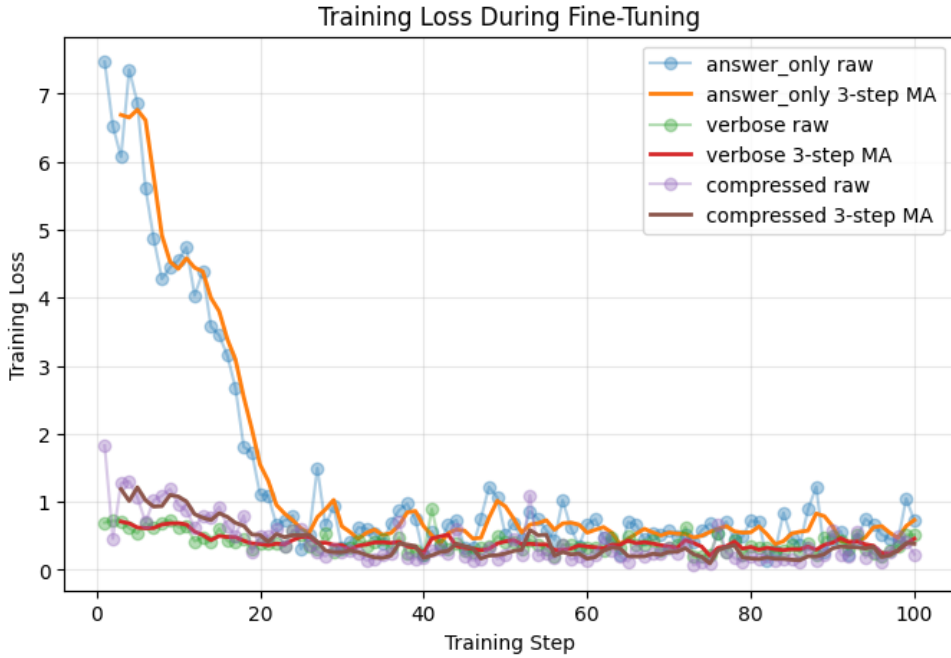


Figure 5: Smoothed training loss during fine-tuning.

These plots should not be over-interpreted. They show that training is running and that the model is learning from the formatted data, but they do not prove that the model has fully converged.

Student Evaluation Results

The main pilot results are summarized in Table 4. The three conditions have very different behavior.

Table 4: Pilot evaluation results across supervision conditions.

Condition	Accuracy	Avg. Latency (s)	Avg. Output Length
Answer Only	14%	1.24	2.00
Verbose CoT	68%	29.27	127.62
Compressed CoT	64%	10.80	41.74

The answer-only model is extremely short and fast, but it only reaches 14% accuracy. This suggests that final answers alone are not enough supervision for this small model. Verbose CoT reaches the highest accuracy at 68%, but it is also the slowest and produces the longest outputs. Compressed CoT reaches 64% accuracy, only four points below verbose CoT, while being much faster and shorter.

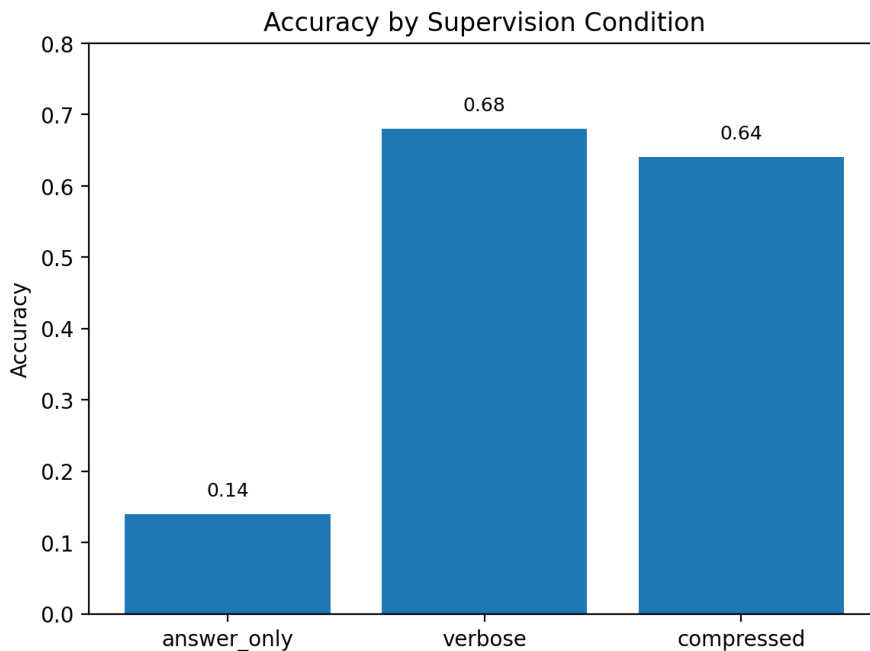


Figure 6: Accuracy by supervision condition.

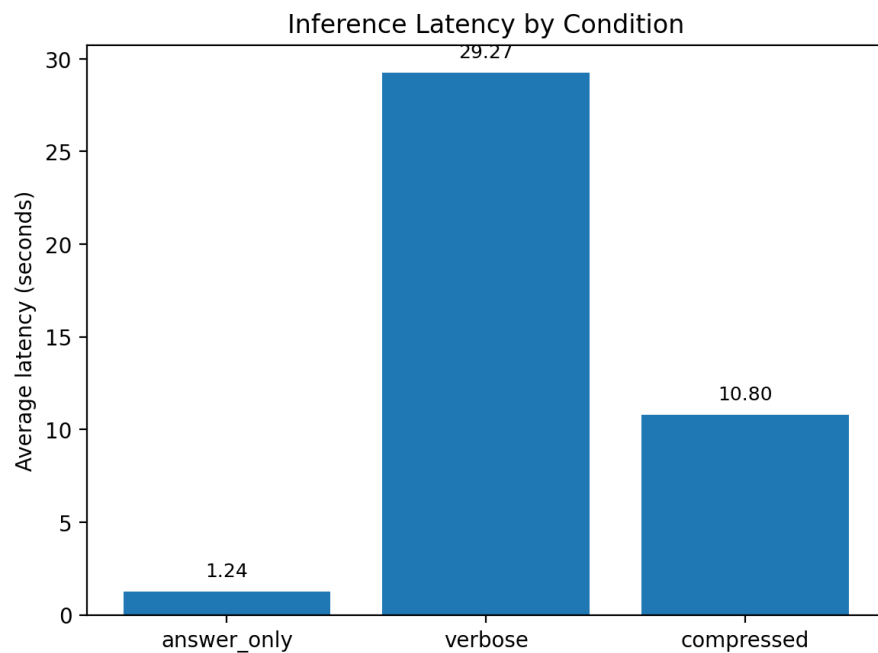


Figure 7: Inference latency by condition.

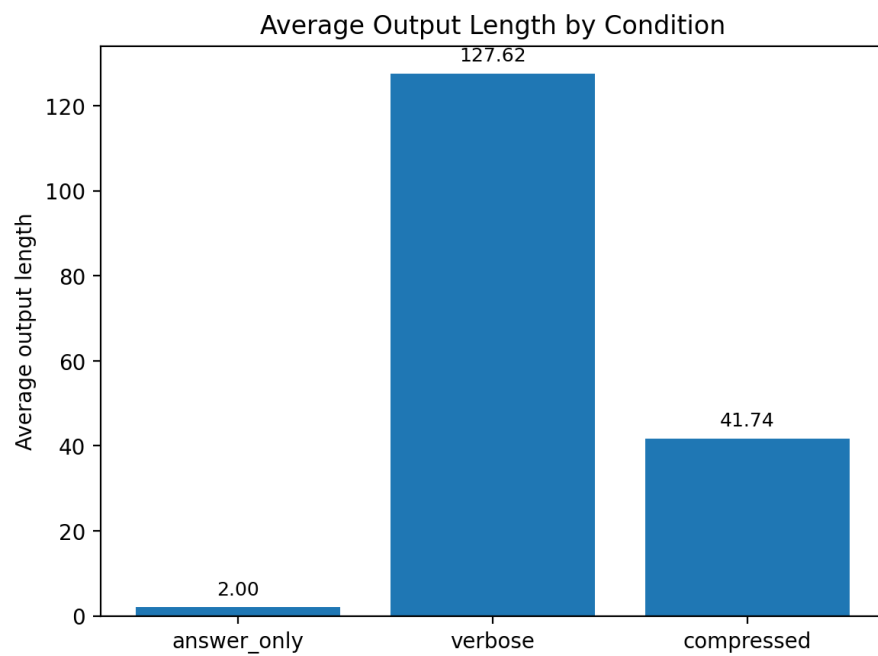


Figure 8: Average output length by condition.

condition	accuracy	avg_latency_sec	avg_output_length
answer_only	0.14	1.241531	2.00
verbose	0.68	29.272839	127.62
compressed	0.64	10.802905	41.74

Figure 9: Summary table of accuracy, latency, and output length.

Compared with verbose CoT, compressed CoT reduces average latency from 29.27 seconds to 10.80 seconds. This is about a $2.7\times$ speedup. It also reduces output length from 127.62 to 41.74, which is about a 67% reduction. The result is not that compressed CoT is always better than verbose CoT. Rather, it gives a better trade-off when latency and output cost matter.

Analysis and Discussion

Why Reasoning Traces Matter

The low performance of the answer-only model is one of the clearest results. It shows that simply giving a small model the final label does not teach it enough about the reasoning process. For GSM8K problems, the answer usually depends on several intermediate computations. If the model never sees those steps during training, it has less guidance about how to solve similar problems.

Both CoT conditions perform much better. Verbose CoT reaches 68% accuracy, and compressed CoT reaches 64%. This suggests that the intermediate reasoning trace is useful as a training signal.

The Accuracy-Efficiency Trade-Off

Verbose CoT is the most accurate condition in the pilot evaluation, but it also has a high cost. It generates long responses and takes much more time per example. Compressed CoT gives slightly lower accuracy, but it is much more efficient.

This is the most important trade-off in the project. In some applications, the extra four percentage points from verbose CoT may be worth the cost. In other applications, especially interactive tutoring or mobile deployment, compressed CoT may be the better choice because it is much faster and still reasonably accurate.

Implications for Educational AI

Compressed CoT is especially relevant for education. A tutoring system does not always need to show a long solution. Often, students benefit from seeing the main arithmetic steps in a concise format. Compressed reasoning can also make smaller models more realistic for classrooms or devices with limited compute. It may also help teacher-facing tools provide faster feedback while still showing some reasoning evidence.

Limitations

There are several limitations. First, the pilot evaluation uses only 20 examples per condition, so the accuracy numbers have high variance. Second, the models are trained for only 50 steps per condition. More training could change the relative performance. Third, the answer extraction rule is simple. If a model gives the correct number in an unexpected format, the current evaluation may count it as wrong. Fourth, the experiment only tests one student model. A larger or different model may respond differently to compressed reasoning. Finally, GSM8K is only one math benchmark, so the results may not transfer directly to other reasoning tasks.

Future Work

The first improvement would be to train on a larger part of GSM8K and evaluate on a larger fixed test set. The second improvement would be to test more training steps and tune LoRA hyperparameters. It would also be useful to test intermediate compression levels instead of only comparing fully verbose and compressed traces.

Another important direction is explanation quality. Accuracy alone does not tell us whether an explanation is useful to a student. Future work could use human ratings or a rubric to judge whether compressed explanations are clear and pedagogically helpful.

Conclusion

This project builds a small end-to-end pipeline for chain-of-thought distillation on GSM8K. We compare answer-only, verbose CoT, and compressed CoT supervision using a Qwen2.5-3B-Instruct student model fine-tuned with LoRA. The pilot results show a clear pattern: answer-only supervision is efficient but weak, verbose CoT is accurate but expensive, and compressed CoT gives a strong balance between the two.

Compressed CoT reaches 64% accuracy compared with 68% for verbose CoT, while reducing latency by about $2.7\times$ and output length by about 67%. The experiment is still limited by small evaluation size and short training, so it should be read as a pilot study. Still, the results support compressed reasoning as a promising direction for efficient mathematical reasoning models.

References

- [1] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*.
- [2] Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., and Schulman, J. (2021). Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- [3] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2021). LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.
- [4] Hsieh, C.-Y., Li, C.-L., Yeh, C.-K., Nakhost, H., Fujii, Y., Ratner, A., Krishna, R., Lee, C.-Y., and Pfister, T. (2023). Distilling step-by-step: Outperforming larger language models with less training data and smaller model sizes. *arXiv preprint arXiv:2305.02301*.

- [5] Qwen Team. (2024). Qwen2.5: A series of large language models. Technical report and model documentation.